GENAI ENGINEERING

A Complete Text-Based Field Guide

A 10,000+ word technical learning reference and RAG evaluation corpus

Topics: LLMs • Transformers • Prompt Engineering • RAG • Embeddings • Chunking • Retrieval • Reranking • Context Engineering • Tool Calling • Agents • FastAPI • Databases • Evaluation • Security • Observability • Deployment • Interview Preparation

Plain text only. Designed for document ingestion and RAG testing.

TABLE OF CONTENTS

GENAI ENGINEERING: A COMPLETE TEXT-BASED FIELD GUIDE

How to Use This Document

1. What Generative AI Actually Is

2. The Core LLM Mental Model

3. Tokens, Embeddings, and Vector Representations

4. Transformer Architecture

5. Prompt Engineering

6. Retrieval-Augmented Generation

7. Document Loading and Data Preparation

8. Chunking Strategies

9. Retrieval: Vector, Keyword, and Hybrid Search

10. Reranking

11. Context Engineering

12. Grounded Generation and Hallucination Control

13. Function Calling and Tool Use

14. Agents and Agentic AI

15. LangChain and LangGraph Concepts

16. FastAPI for GenAI Backends

17. Databases for GenAI Applications

18. Fine-Tuning vs RAG vs Prompting

19. RAG Evaluation

20. Observability and Debugging

21. Security for GenAI Systems

22. Responsible and Reliable GenAI

23. Caching, Cost, and Performance

24. Deployment and MLOps for GenAI

25. Testing Strategy

26. A Production-Grade GenAI Project Blueprint

27. Interview Questions a GenAI Engineer Should Master

28. A Practical Learning Roadmap

29. Portfolio Project Ideas

30. Final Engineering Principles

31. Extended Technical Glossary

32. RAG Test Question Bank

33. Suggested RAG Evaluation Dataset Design

34. Conclusion: What a GenAI Engineer Really Builds

Appendix A. Detailed RAG Failure Analysis

Appendix B. Example Production Request Lifecycle

Appendix C. Advanced Retrieval Design

Appendix D. Advanced Agent Architecture and Guardrails

Appendix E. Data Governance and Knowledge-Base Lifecycle

Appendix F. Cost Modeling for a GenAI Application

Appendix G. System Design Interview Framework

Appendix H. Practical Engineering Checklist

How to Use This Document

This guide is written for two purposes. First, it is a learning reference for someone who wants to become a Generative AI Engineer rather than merely learn how to call an LLM API. Second, it is a useful corpus for testing document ingestion, chunking, embeddings, vector retrieval, metadata filtering, reranking, context construction, generation, citation, and evaluation in a RAG application.

The document intentionally contains closely related concepts that are easy to confuse. For example, semantic search, vector search, hybrid search, reranking, and RAG are discussed separately. Prompt engineering, tool calling, workflows, agents, and agentic systems are also separated. This gives a retrieval system meaningful opportunities to distinguish related concepts rather than relying on exact keyword matching.

When using this document as a RAG dataset, questions should test both direct retrieval and synthesis. Good evaluation questions include: What is the difference between an embedding model and a generative model? Why can a high retrieval score coexist with poor answer quality? When should a system use hybrid search instead of vector-only search? What is the difference between an agent and a deterministic workflow? Which safeguards are appropriate for tool-using systems? The best RAG evaluation set contains both easy factual questions and questions that require connecting information from multiple sections.

1. What Generative AI Actually Is

Generative artificial intelligence refers to models and systems capable of producing new content conditioned on an input. The generated content can be text, code, images, audio, video, structured data, or combinations of these modalities. In modern software engineering, the most visible form is the large language model, or LLM, which generates text or structured token sequences from a prompt.

The word generative is important. A traditional classifier may map an input to one of several labels. A retrieval system may return existing records. A generative model instead learns statistical patterns from large collections of data and uses those patterns to construct a new sequence. This does not mean that every generated statement is true. Generation is a probabilistic modeling process, while factual correctness is a separate engineering requirement.

Modern LLM applications are best understood as systems rather than as isolated models. A production GenAI application can include an API layer, authentication, a prompt construction layer, a model provider, retrieval infrastructure, databases, tool integrations, safety checks, observability, evaluation pipelines, caching, rate limiting, and a user interface. The model is only one component.

A useful mental model is: input -> preprocessing -> context selection -> prompt construction -> model inference -> validation -> output formatting -> logging and evaluation.

This architecture immediately explains why learning only prompting is insufficient for an engineering career. An engineer must understand data, APIs, latency, security, failure modes, testing, deployment, and cost. The model is powerful, but the surrounding software determines whether the system is reliable.

Generative AI should also be separated from the broader field of machine learning. Generative models are a subset of machine learning models. LLMs are a subset of generative models. A GenAI engineer is therefore not necessarily a research scientist who trains foundation models from scratch. In many industry roles, the engineer integrates existing models into reliable applications, adapts them with retrieval or fine-tuning when appropriate, and builds the infrastructure required for production use.

2. The Core LLM Mental Model

A language model estimates the likelihood of token sequences. Text is converted into tokens, and the model predicts the next token conditioned on previous tokens. A token is not always a whole word. It may represent a

word, part of a word, punctuation, whitespace pattern, or another unit chosen by the tokenizer.

During inference, the model repeatedly selects or samples tokens until it reaches a stopping condition. Temperature influences how strongly the generation process favors high-probability choices. Lower temperature generally makes output more deterministic; higher temperature permits more variation. Temperature is not a factuality control. A low-temperature answer can still be confidently wrong.

Top-p and top-k are sampling controls. Top-k restricts candidate selection to a fixed number of likely tokens, while top-p selects from the smallest probability mass whose cumulative probability reaches a chosen threshold. Exact behavior and supported parameters vary by model provider.

Context length describes how much tokenized information a model can process within a request according to its architecture and provider limits. A larger context window does not automatically make an application better. Feeding irrelevant documents increases cost, latency, and the opportunity for the model to attend to distracting information. Context quality often matters more than raw context quantity.

Inference cost is affected by input and output tokens, model pricing, provider, caching behavior, and sometimes additional tool or retrieval operations. A production engineer therefore treats token usage as an engineering metric.

An LLM also does not inherently know which information is current. Its learned parameters reflect training data and model updates, while external knowledge can change daily. RAG, tool calling, search, databases, and APIs are common methods for connecting a model to current or private information.

Another important distinction is knowledge versus context. Parameters store learned statistical representations. Context is information supplied at inference time. A RAG system does not normally rewrite the model's parameters when a document is retrieved. Instead, it inserts relevant text into the model's input so the model can use that information while generating the answer.

3. Tokens, Embeddings, and Vector Representations

Tokenization converts text into a sequence of discrete identifiers. The exact tokenization strategy depends on the model family. Engineers should understand tokenization because it affects context length, cost, truncation, prompt design, and document chunk size.

An embedding is a numerical vector representation of an input such as text. Embedding models are trained so that semantically related inputs tend to have representations that are useful for similarity search or other downstream tasks. The dimensions of an embedding vector depend on the model.

Similarity can be measured in several ways. Cosine similarity compares the angle between vectors and is common for normalized semantic embeddings. Dot product compares vector alignment and magnitude. Euclidean distance measures geometric separation. The appropriate metric depends on the embedding model and vector database configuration.

Embedding quality is task-dependent. A model that performs well for general semantic search may not be optimal for legal clauses, source code, multilingual content, product catalogs, or highly technical documents. Language coverage, domain vocabulary, query-document asymmetry, vector dimension, latency, and licensing all matter.

A common RAG flow is: 1. Load source documents. 2. Split documents into chunks. 3. Generate an embedding for each chunk. 4. Store the vectors with the original text and metadata. 5. Embed a user query using the same compatible embedding space. 6. Search for nearby vectors. 7. Pass selected chunks to the generation model.

The same embedding model should normally be used for indexing and querying unless a retrieval system is explicitly designed around compatible asymmetric encoders. Changing the embedding model after indexing requires careful consideration because vectors generated by different models generally do not share the same representation space.

Vector search is not a magic semantic database. It returns items that are close according to the chosen representation and metric. If the source text is poorly chunked, duplicated, stale, or missing important metadata, excellent embeddings cannot completely repair the pipeline.

4. Transformer Architecture

The Transformer architecture is the foundation of most modern LLMs. Its central mechanism is attention, which allows the model to compute relationships between token representations. Instead of processing a sequence only through a strictly sequential recurrent state, attention enables information from different positions to interact.

Self-attention uses queries, keys, and values. For each token representation, the model produces these vectors and computes attention scores that determine how strongly one position attends to another. Multiple attention heads allow different representational subspaces to capture different relationships.

Transformer blocks typically contain attention, feed-forward transformations, normalization, residual connections, and other model-specific components. Decoder-style language models are commonly trained with causal masking, meaning a token cannot attend to future tokens during next-token prediction.

Pretraining is the large-scale stage in which a model learns general statistical structure from a broad corpus. A common objective for decoder language models is next-token prediction. The model is exposed to enormous numbers of token sequences and learns parameters that reduce prediction error.

Post-training can include supervised fine-tuning, preference optimization, reinforcement learning methods, instruction tuning, safety training, and other techniques. The terminology and exact algorithms vary across model families.

A useful engineering distinction is that pretraining teaches broad capabilities and representations, while post-training shapes behavior, instruction following, style, safety, and task performance. Application developers generally consume a pretrained and post-trained model rather than performing foundation-scale pretraining themselves.

Model size is not the only determinant of application quality. Smaller models can be attractive when latency, cost, privacy, or on-device execution matter. A larger model may perform better on difficult reasoning tasks but cost more and require more infrastructure. Model selection should therefore be based on measured task performance and operational constraints rather than parameter count alone.

5. Prompt Engineering

Prompt engineering is the deliberate design of model inputs to improve the reliability and usefulness of generated outputs. It includes instruction design, role definition, task decomposition, examples, constraints, output schemas, context formatting, and failure handling.

A strong prompt usually makes the task explicit. It defines what the model should do, what information it may use, what format it should return, and what it should do when the required information is absent.

Few-shot prompting provides examples of desired input-output behavior. Examples can teach a format, classification rule, style, or reasoning pattern. However, examples consume context and can introduce bias or accidental constraints.

Structured output is particularly valuable in software systems. Instead of asking for arbitrary prose, an application can request a schema containing fields such as answer, confidence, sources, and action. When supported by the model and framework, schema-constrained generation can make downstream parsing more reliable.

Prompt injection is a major security issue. If an application retrieves documents or accepts user content, those inputs may contain instructions that conflict with the system's intended behavior. Retrieved text should therefore be treated as untrusted data, not automatically as higher-priority instructions.

Prompt design is not a substitute for system architecture. If a model repeatedly invents database records, the correct solution may involve tool calls or deterministic database queries rather than adding more words to the prompt. If a RAG system retrieves irrelevant chunks, retrieval must be improved rather than attempting to solve the problem only with generation instructions.

A mature engineer treats prompts as versioned application assets. Prompt changes should be tested, reviewed, and measured. Important production prompts should have regression tests covering normal, ambiguous, adversarial, and missing-information cases.

6. Retrieval-Augmented Generation

Retrieval-Augmented Generation, commonly called RAG, combines information retrieval with language generation. The central idea is simple: retrieve relevant external information and provide it to the generation model as context.

A basic RAG pipeline has two stages. The indexing stage prepares a knowledge corpus. The query stage retrieves relevant information for a user question.

Indexing commonly involves document loading, cleaning, chunking, metadata extraction, embedding, and vector storage. Query-time processing commonly involves query normalization, retrieval, optional reranking, context assembly, prompt construction, generation, and response validation.

RAG is useful when the knowledge is private, changing, domain-specific, too large to place directly in every prompt, or expected to be traceable to source documents. It can also reduce the need to fine-tune a model simply to make factual documents available.

RAG does not guarantee factuality. If the retriever misses the relevant passage, the generator may lack the evidence required to answer. If the retrieved context contains conflicting documents, the model may choose incorrectly. If the prompt does not clearly constrain evidence use, the model may combine retrieved information with unsupported prior knowledge.

The phrase "RAG quality" therefore hides several distinct problems: - ingestion quality, - chunking quality, - embedding quality, - retrieval quality, - reranking quality, - context packing quality, - generation quality, - citation quality, - evaluation quality.

A production RAG system should measure these separately where possible.

7. Document Loading and Data Preparation

Document ingestion is often underestimated. A RAG system can only retrieve what it successfully extracts and indexes. PDF extraction may lose reading order, tables, headers, footers, or scanned text. HTML may contain navigation menus and repeated boilerplate. Office documents may contain structured content that needs specialized parsing.

A robust ingestion pipeline identifies document type, extracts content, normalizes whitespace, removes unwanted repetition, preserves meaningful structure, and attaches metadata. Useful metadata can include document ID, title, author, source path, page number, section heading, creation date, version, department, access classification, and content type.

Metadata is not decoration. It can be used for filtering. For example, a user might only be authorized to retrieve documents belonging to a particular organization. A retrieval query might also restrict results to a product version or date range.

Duplicate documents create another problem. If the same passage appears in many records, the retriever may return several copies, reducing context diversity. Deduplication can be performed using document hashes, normalized text fingerprints, or chunk-level similarity methods.

Versioning matters in enterprise knowledge bases. If a policy changes, the system should know whether older versions remain searchable and whether users should be shown the latest approved version. Blindly adding every new document can create contradictory retrieval results.

A strong ingestion system should be repeatable. Running the ingestion process twice should not create uncontrolled duplicates. Document IDs, chunk IDs, and embedding records should be deterministic or managed through an explicit upsert strategy.

8. Chunking Strategies

Chunking determines how source documents are divided into retrievable units. It is one of the most important RAG design choices because retrieval occurs at the chunk level.

Fixed-size chunking splits text according to a character or token budget, optionally with overlap. It is simple and predictable. Recursive character splitting attempts to break text at increasingly smaller structural boundaries, such as paragraphs and sentences, before falling back to smaller units.

Semantic chunking attempts to identify topic boundaries based on meaning. It can preserve coherent ideas but may be more computationally expensive and can behave unpredictably on unusual documents.

Structure-aware chunking uses document structure such as headings, sections, list items, code blocks, tables, and paragraphs. For technical and enterprise documents, this can be highly effective because the structure often reflects the information architecture.

Chunk size creates a tradeoff. Very small chunks improve precision but may remove necessary context. Very large chunks preserve context but can reduce retrieval precision and waste context-window space. There is no universal best chunk size. The correct value depends on document structure, query type, embedding model, and generation requirements.

Overlap can help prevent information from being split at boundaries. However, excessive overlap creates duplicate information and increases storage and retrieval redundancy.

A useful strategy is to store both the chunk text and its structural metadata. For example, a chunk may carry the title "Access Control", section "Authentication", page 12, and document ID "SEC-2026-014". This makes citations and filtering easier.

Chunking should be evaluated empirically. Create representative questions, identify the expected source chunks, and measure whether retrieval returns those chunks. Do not select chunk size solely because it is common in a tutorial.

9. Retrieval: Vector, Keyword, and Hybrid Search

Vector retrieval finds semantically similar chunks. Keyword retrieval searches exact or lexical terms. Each has strengths and weaknesses.

Vector search can match paraphrases. A query such as "How do I terminate a user account?" may retrieve a document saying "deprovision employee access" even if the exact words differ.

Keyword search is strong for exact identifiers, product codes, error messages, legal terms, names, and rare technical strings. A semantic embedding may not preserve the exact importance of a unique identifier.

Hybrid search combines lexical and semantic signals. A common design retrieves candidates from both systems and combines their scores or candidate sets. Hybrid retrieval is particularly useful in enterprise search where

users ask both conceptual and exact-match questions.

Top-k retrieval selects the k highest-ranked candidates. A small k may miss supporting evidence; a large k may introduce irrelevant context. The optimal value should be measured.

Maximum Marginal Relevance, or MMR, attempts to balance relevance with diversity. Instead of selecting multiple highly similar chunks, it can prefer chunks that are relevant but provide different information. This can be useful when a document contains repetitive sections.

Filtering before retrieval can improve both relevance and security. For example, a system may filter by tenant ID, document type, language, date, or access scope. Security filters must be enforced server-side; they should never depend only on instructions to the LLM.

Retrieval should be observable. Log query IDs, retrieval method, candidate IDs, scores, metadata, and latency where privacy policy permits. Without retrieval traces, debugging a poor answer becomes much harder.

10. Reranking

Reranking is a second-stage retrieval process. An initial retriever produces a candidate set, and a more expensive model evaluates the relationship between the query and each candidate.

A common architecture is: query -> retrieve 20 to 100 candidates -> rerank -> keep top 3 to 10 -> generate.

The first stage is optimized for speed and broad recall. The reranker is optimized for more precise ordering. Cross-encoder models can examine the query and candidate text jointly, often improving ranking quality over simple embedding similarity.

Reranking is useful when the vector database returns several plausible passages but the top few are not ordered correctly. It is especially valuable for long or heterogeneous corpora.

Reranking has a cost. Running a sophisticated model over many candidates increases latency and compute. A production system therefore needs a candidate limit and a clear latency budget.

Reranking cannot recover a document that was never retrieved. This leads to an important distinction: retrieval recall is a prerequisite for reranking success. If the relevant chunk is outside the candidate set, a perfect reranker has nothing useful to rank.

A good evaluation pipeline therefore measures both first-stage recall and post-reranking ranking quality. The system should not be judged only by the final generated answer.

11. Context Engineering

Context engineering is the broader practice of deciding what information enters the model context, how it is ordered, labeled, compressed, and constrained. It is closely related to RAG but is broader than retrieval alone.

Retrieved passages should be clearly separated from instructions. A useful prompt can include a system instruction, user question, retrieved context, source identifiers, and output requirements.

Ordering can matter because models do not necessarily use every context position equally. A practical system should test whether placing the most relevant evidence first, last, or in a structured format changes answer quality.

Context compression can reduce token usage. It may involve extracting relevant sentences, summarizing candidate passages, or using a model to remove irrelevant material. Compression can also remove evidence, so it must be evaluated carefully.

Context packing should avoid redundant chunks. If five retrieved chunks say nearly the same thing, the model receives less diverse evidence than if those slots contain complementary sections.

For long-context systems, "retrieve everything and let the model figure it out" is not automatically better. Long context can increase cost and latency, and irrelevant information can interfere with the answer.

Context should be treated as a limited engineering resource. The objective is not maximum text; it is maximum useful evidence per token.

12. Grounded Generation and Hallucination Control

A hallucination is an output that contains unsupported, fabricated, or incorrect information. In a RAG application, hallucination control means designing the system so answers are grounded in available evidence and unsupported claims are handled safely.

A basic grounding instruction might require the model to answer using supplied context and state when the context is insufficient. However, instructions alone are not sufficient. The application should also improve retrieval, provide source identifiers, validate outputs, and evaluate groundedness.

Faithfulness asks whether the answer is supported by the retrieved context. Context relevance asks whether the retrieved context is actually useful for the question. These are different metrics. A model can produce a faithful answer to irrelevant documents, or receive relevant documents and still invent unsupported statements.

A useful refusal behavior is not a generic refusal to answer everything. The system should distinguish between "the answer is present in the evidence" and "the evidence is insufficient." In the second case, it can say that the available knowledge base does not contain enough information.

Citations improve traceability but are not automatically proof of correctness. A model can attach a real citation to an unsupported statement. Citation correctness must therefore be evaluated.

For high-stakes applications, deterministic validation should be used whenever possible. For example, a financial balance should come from a database calculation rather than an LLM's arithmetic. A medical or legal system should apply appropriate domain-specific safeguards and professional review.

13. Function Calling and Tool Use

Tool calling allows a model to request execution of a predefined function. The application receives structured arguments, validates them, executes the tool, and returns the result to the model.

Tools can include database queries, search, calculators, ticket systems, CRM operations, calendars, code execution environments, and internal APIs.

The key engineering principle is that the LLM should not directly control arbitrary infrastructure. Tools should expose narrow, validated operations. For example, a tool named get_customer_balance may accept a customer ID and return a read-only balance. It is safer and easier to audit than giving the model unrestricted database access.

Tool schemas should specify argument types and constraints. The server must validate arguments again even if the model generated structured output. Never treat model-generated arguments as trusted input.

Side-effecting tools deserve additional safeguards. Sending an email, deleting a record, issuing a refund, or deploying code may require user confirmation, authorization checks, idempotency controls, and audit logs.

Tool execution should have timeouts, retries where safe, and clear error handling. A model should be able to receive a structured tool error without causing an uncontrolled loop.

Tool calling is not identical to an agent. A single model response can call one tool deterministically. An agentic system usually involves iterative decision-making, where the model decides which action to take next based on observations.

14. Agents and Agentic AI

An agentic system is a software system in which a model participates in a loop of planning, action, observation, and further decision-making. The exact definition of agent varies across frameworks, so it is useful to focus on behavior rather than terminology.

A deterministic workflow has a predefined sequence: retrieve -> summarize -> classify -> respond.

An agent may instead receive a goal and decide: search -> inspect result -> call database -> compare -> ask another tool -> produce answer.

Agents are powerful because they can adapt their path to the problem. They are also harder to test, debug, secure, and control. More autonomy creates more possible failure paths.

A production agent should have explicit boundaries. These include a maximum number of iterations, tool allowlists, execution timeouts, token budgets, state limits, permission checks, and escalation conditions.

Agent memory can mean several different things. Conversation history is short-term context. Persistent user preferences are long-term memory. A vector database is not automatically "memory"; it is a retrieval mechanism that can be used to implement a memory-like capability.

Agent state should be explicit. Frameworks such as graph-based orchestration systems can represent state transitions and conditional branches. This makes complex workflows easier to inspect than an uncontrolled while-loop.

Not every problem needs an agent. If the workflow is predictable, a deterministic pipeline is often simpler and easier to operate. Use autonomy where adaptation provides measurable value.

15. LangChain and LangGraph Concepts

LangChain is a framework ecosystem for composing language-model applications. Common building blocks include model interfaces, prompt templates, document loaders, text splitters, retrievers, output parsers, and tool integrations.

A chain generally represents a composition of operations. A retriever can be connected to prompt construction and model generation to create a RAG pipeline.

LangGraph is designed for stateful, graph-based workflows and agentic applications. Nodes perform work, edges determine transitions, and state carries information through the graph.

A graph model can represent a workflow such as: START -> classify question -> if factual, retrieve -> generate -> validate -> END. Another branch could handle requests requiring a tool.

The engineering advantage of graph orchestration is visibility. Complex agent behavior can be represented as explicit nodes and transitions rather than hidden inside a long prompt.

Framework abstractions are useful, but engineers should understand the underlying HTTP requests, model messages, embeddings, vector database operations, and Python control flow. A developer who only knows framework syntax can struggle when a provider changes an API or when a performance problem occurs.

Production systems should avoid unnecessary abstraction. If a simple direct model call solves the problem, adding multiple framework layers may increase complexity without adding value.

16. FastAPI for GenAI Backends

FastAPI is a Python web framework commonly used to expose GenAI functionality through HTTP APIs. It is well suited to applications that need typed request models, asynchronous endpoints, dependency injection,

automatic OpenAPI documentation, and integration with Python AI libraries.

A production GenAI backend may expose endpoints such as: POST /chat POST /documents POST /search POST /feedback GET /health GET /documents/{document_id}

Authentication and authorization should be enforced at the API layer. A user request should carry identity and tenant information that the backend can validate before retrieving data.

Streaming is important for conversational applications because users often perceive a response as faster when tokens are delivered incrementally. Server-Sent Events or WebSockets can be used depending on the interaction pattern and infrastructure.

Long-running tasks such as document ingestion should generally be separated from request-response latency. A background worker or task queue can process files while the API reports job status.

FastAPI does not make an AI system production-ready by itself. The application still needs logging, timeouts, retries, rate limiting, secret management, monitoring, tests, and deployment practices.

17. Databases for GenAI Applications

GenAI systems commonly use several database types at once. A relational database can store users, documents, permissions, conversations, feedback, and business records. A vector store can support semantic retrieval. Object storage can hold source files.

PostgreSQL is a common relational choice. It supports transactions, indexes, constraints, joins, and extensions that can also support vector workloads in suitable architectures.

Vector databases are optimized for similarity search over embedding vectors and associated metadata. Examples in the ecosystem include Chroma, FAISS-based systems, and specialized hosted vector databases. The correct choice depends on scale, persistence requirements, filtering, operational constraints, and team expertise.

FAISS is primarily a similarity-search library rather than a complete enterprise database. It can be excellent for local experiments and high-performance vector indexing but requires application-level handling for many database concerns.

Chroma provides a developer-friendly vector-store experience and is convenient for prototyping and smaller applications. Production selection should still consider persistence, concurrency, backup, security, observability, and operational support.

A common architecture separates source-of-truth data from derived embeddings. The original document remains authoritative, while chunks and vectors can be regenerated when the embedding model or chunking strategy changes.

18. Fine-Tuning vs RAG vs Prompting

Prompting changes the instructions supplied at inference time. RAG supplies external knowledge at inference time. Fine-tuning changes model parameters through additional training.

Use prompting when the problem is primarily behavior, formatting, instruction following, or task definition.

Use RAG when the model needs access to external, private, dynamic, or traceable information.

Consider fine-tuning when a model needs a learned behavioral or domain adaptation that is difficult to achieve reliably through prompting alone and when enough high-quality training examples exist.

These methods can be combined. A customer-support system may use a fine-tuned model for response style, RAG for current product documentation, and tool calling for account-specific information.

Fine-tuning should not be treated as a method for continuously updating factual knowledge. If facts change frequently, retrieval or database access is usually more operationally appropriate.

The choice should be based on measurable requirements: accuracy, latency, cost, privacy, update frequency, data availability, and maintenance burden.

19. RAG Evaluation

Evaluation is one of the most important differences between a demonstration and a serious GenAI system. A chatbot that appears good in a few manual tests is not necessarily reliable.

Evaluation should include a representative dataset of questions and expected evidence. A test record can contain the question, expected answer characteristics, relevant document IDs, relevant chunk IDs, and acceptable alternative answers.

Retrieval metrics include recall at k, precision at k, mean reciprocal rank, and related ranking measures. Recall at k asks whether the required evidence appears among the top k retrieved items.

Generation metrics can include correctness, relevance, groundedness, and citation accuracy. Automated LLM-as-judge methods can be useful but should not be treated as unquestionable ground truth. Human review is still important for a carefully selected sample.

RAG evaluation frameworks can help measure dimensions such as faithfulness, context relevance, context recall, and answer relevance. The exact metric definitions and implementation details differ by framework.

A particularly useful debugging matrix is: retrieval correct + answer wrong = generation/context problem; retrieval wrong + answer wrong = retrieval problem; retrieval correct + answer correct = successful case; retrieval wrong + answer apparently correct = potentially lucky or parametric-knowledge answer.

The last category matters because a model can answer correctly without using the retrieved evidence. If the application promises knowledge-base grounding, such behavior should be visible in evaluation.

20. Observability and Debugging

Observability means collecting enough information to understand what the system did. A useful GenAI trace can include request ID, user or tenant identifier where appropriate, model name, prompt version, retrieval query, retrieved document IDs, retrieval scores, reranker scores, token counts, latency, tool calls, errors, and final response metadata.

Privacy must be considered when logging prompts and documents. Sensitive data should not be copied into logs unnecessarily. Access to traces should be restricted.

Latency should be broken into components. For a RAG request, measure embedding latency, vector-search latency, reranking latency, prompt construction time, model time-to-first-token, total generation time, and tool execution time.

Cost observability should track input and output tokens and provider pricing. Cached results can reduce cost, but caching must respect authorization and data freshness.

Common debugging questions include: Did the correct document enter the index? Was the document parsed correctly? Was the relevant content split across chunks? Did the query embedding work? Did retrieval return the expected chunk? Did reranking reorder it correctly? Was the chunk inserted into the prompt? Did the model follow the grounding instruction? Was the final citation attached to the correct source?

Without these traces, developers often modify prompts blindly.

21. Security for GenAI Systems

GenAI applications introduce familiar application-security risks plus model-specific risks. These include prompt injection, sensitive data exposure, insecure tool use, excessive permissions, data poisoning, model denial of service, insecure output handling, and supply-chain risks.

Prompt injection occurs when untrusted content attempts to influence model behavior. Retrieved documents should therefore be treated as data. System instructions and authorization logic should remain outside the model's control.

Sensitive information should be minimized. Do not send unnecessary personal or confidential data to a third-party model provider. Use appropriate encryption, access controls, retention policies, and provider agreements where required.

Tool permissions should follow least privilege. A support agent that only needs to read customer data should not have permission to delete accounts.

Output is also untrusted. If generated text is inserted into HTML, SQL, shell commands, or another interpreter without proper handling, the application can become vulnerable. The correct defense is parameterization, escaping, allowlisting, validation, and strict API boundaries rather than asking the LLM to "be safe."

Multi-tenant RAG systems require strict tenant isolation. A retrieval query must never accidentally return another tenant's chunks. Authorization filtering should occur before sensitive context reaches the model.

Security testing should include malicious prompts, unauthorized tool requests, cross-tenant retrieval tests, poisoned documents, malformed inputs, excessive requests, and failure recovery.

22. Responsible and Reliable GenAI

Responsible AI engineering involves fairness, privacy, transparency, safety, accountability, and appropriate human oversight. The required controls depend heavily on the use case.

For low-risk content drafting, a user may review generated text before publication. For higher-risk applications, the system may require stronger validation, human approval, restricted actions, and documented escalation paths.

Models can reproduce biases present in training data. Testing should therefore consider relevant populations and failure modes rather than relying only on average accuracy.

A reliable system also communicates uncertainty appropriately. It should not manufacture confidence scores unless those scores have been validated. "I do not have enough information" can be a valuable system behavior.

Human-in-the-loop design is especially important for consequential actions. The system can prepare a recommendation, but a qualified human may need to approve an external action.

Responsible deployment is not a single prompt. It is a system property created by model choice, data governance, access control, evaluation, monitoring, user experience, and operational procedures.

23. Caching, Cost, and Performance

Performance engineering in GenAI involves latency, throughput, memory, concurrency, and cost.

Caching can be applied at multiple levels. Embedding caching avoids recomputing vectors for repeated text. Retrieval caching can reuse search results for repeated queries when the underlying corpus has not changed. Model-response caching can avoid repeated generation for identical requests when the application can tolerate cached responses.

Caching must account for authorization. A response generated for one user or tenant must not leak to another through a shared cache.

Batching can improve throughput for embedding jobs. Asynchronous processing can prevent slow external model calls from blocking unrelated requests.

Model selection is a major cost lever. A smaller model may be sufficient for classification, query rewriting, extraction, or simple transformations, while a more capable model is reserved for difficult reasoning.

Prompt size is another cost lever. Retrieving ten large chunks when three precise chunks are sufficient can increase both cost and latency.

A mature system measures cost per request, latency percentiles, error rate, retrieval quality, and user-visible success. Optimizing only token price can be counterproductive if a cheaper model causes higher failure rates and more retries.

24. Deployment and MLOps for GenAI

A production GenAI application needs a deployment strategy for application code, prompts, model configuration, vector indexes, documents, and evaluation datasets.

Containerization can package the API and its dependencies consistently. Docker is commonly used for this purpose.

Environment variables or a secret-management system should store API keys and credentials. Secrets should never be committed to source control.

Continuous integration should run unit tests, integration tests, linting, and selected evaluation tests. A prompt or model change can alter behavior even when Python tests pass, so regression evaluation is important.

Deployment should support rollback. Model providers, prompts, retrieval settings, and application code can all change system behavior.

Document indexing should be treated as a deployment process. A new corpus version should be identifiable, reproducible, and reversible when possible.

Health checks should distinguish basic application availability from dependency availability. A service can be alive while its vector database or model provider is unavailable.

Production monitoring should cover uptime, latency, error rates, token usage, cost, retrieval quality, tool failures, and user feedback.

25. Testing Strategy

GenAI testing should combine deterministic software tests with probabilistic evaluation.

Unit tests can validate chunking, metadata extraction, authorization filters, prompt formatting, parser behavior, and tool argument validation.

Integration tests can verify the complete path from API request to retrieval and model invocation using controlled fixtures.

Evaluation tests can use a fixed question set to detect regressions in retrieval and generation quality. The expected answer may be a set of acceptable facts rather than an exact string.

Adversarial tests should include ambiguous questions, irrelevant questions, malformed inputs, prompt injection attempts, missing documents, contradictory documents, oversized inputs, and unauthorized requests.

For RAG specifically, test whether known evidence is retrieved. Do not evaluate only final answer text. If a final answer happens to be correct because the model already knew the fact, the retrieval pipeline may still be broken.

For tool-using agents, test invalid arguments, tool failures, repeated failures, timeouts, permission denials, and side-effect confirmation behavior.

A strong regression suite grows from production incidents. Every meaningful failure should become a test case so the same class of bug is less likely to return.

26. A Production-Grade GenAI Project Blueprint

A strong portfolio project should demonstrate more than a chatbot interface. A production-oriented knowledge assistant can include document ingestion, structured metadata, hybrid retrieval, reranking, grounded generation, citations, authentication, FastAPI endpoints, persistent storage, evaluation, logging, Docker deployment, and a simple user interface.

A reasonable architecture is: Client -> FastAPI -> authentication -> query processing -> retrieval -> reranker -> context builder -> LLM -> citation validator -> response.

The ingestion path is separate: file upload -> parser -> cleaner -> structure-aware chunker -> metadata -> embeddings -> vector store.

A relational database can store users, document records, ingestion jobs, conversations, and feedback. The vector store stores chunk embeddings and searchable metadata. Object storage can retain original files.

The project should include an evaluation dataset with at least several dozen representative questions. Each question should identify relevant documents or chunks where possible.

The README should explain architecture decisions and tradeoffs. It should include why a particular embedding model was chosen, why a chunking strategy was selected, how retrieval was evaluated, what failure cases remain, and how the system handles unsupported questions.

A portfolio project becomes much stronger when it demonstrates engineering judgment rather than simply listing technologies.

27. Interview Questions a GenAI Engineer Should Master

A GenAI engineer should be able to explain what happens between a user question and a final answer. They should understand tokenization, embeddings, vector search, hybrid retrieval, reranking, prompt construction, context windows, model inference, tool calling, and evaluation.

Typical conceptual questions include: Why does RAG reduce the need to put all company knowledge into model parameters? Why can increasing top-k reduce answer quality? What is the difference between cosine similarity and dot product? Why is a reranker useful after vector retrieval? When is keyword search stronger than semantic search? Why can a model hallucinate even when relevant documents are present? What is the difference between fine-tuning and RAG? How would you debug a RAG system that returns irrelevant answers? How would you prevent one tenant from retrieving another tenant's documents? What happens if a tool call times out? Why is an agent not always better than a workflow?

Strong interview answers should explain tradeoffs, not merely definitions. For example, if asked about chunk size, do not provide one universal number. Explain that chunk size depends on document structure, query type, embedding model, and context constraints, and then describe how you would evaluate alternatives.

For a project discussion, be prepared to explain one difficult failure you encountered, how you measured it, what change you made, and what metric improved. This demonstrates engineering thinking.

28. A Practical Learning Roadmap

Phase 1 should establish strong Python fundamentals, including functions, classes, exceptions, typing, asynchronous programming, virtual environments, package management, testing, and Git.

Phase 2 should cover backend engineering: HTTP, REST APIs, FastAPI, Pydantic, authentication, authorization, PostgreSQL, SQLAlchemy, Docker, logging, and deployment basics.

Phase 3 should cover ML and NLP fundamentals: tokens, embeddings, similarity, transformers, attention, model inference, evaluation, and the distinction between training and inference.

Phase 4 should focus on LLM application development: prompt engineering, structured output, function calling, streaming, model APIs, error handling, cost control, and observability.

Phase 5 should focus on RAG: document loading, cleaning, chunking, embeddings, vector databases, metadata filters, hybrid retrieval, reranking, context engineering, citations, and RAG evaluation.

Phase 6 should cover agents and workflows: state machines, tool use, graph orchestration, memory, guardrails, human approval, and agent evaluation.

Phase 7 should focus on production: Docker, CI/CD, cloud deployment, security, monitoring, testing, load testing, secrets, data governance, and incident response.

The best learning strategy is project-driven. Build a small system, measure its weaknesses, improve one component, and document the result. Repeating this cycle produces deeper understanding than watching many disconnected tutorials.

29. Portfolio Project Ideas

Project 1: Enterprise Knowledge Intelligence System. Build a RAG assistant over company policies, technical manuals, onboarding documents, and product documentation. Include metadata filtering, reranking, citations, evaluation, and FastAPI.

Project 2: AI Customer Support Agent. Combine RAG for product knowledge with tools for order lookup, ticket creation, and escalation. Add authentication and human approval for side effects.

Project 3: Multi-Agent Research Workflow. Use separate components for query planning, retrieval, source analysis, synthesis, and verification. Keep the workflow constrained and observable rather than creating an uncontrolled collection of agents.

Project 4: Codebase Intelligence Assistant. Index a software repository with structure-aware chunks, retrieve relevant files, explain dependencies, and provide source references. Evaluate whether retrieved code actually contains the evidence used in answers.

Project 5: Document Intelligence API. Upload PDFs, extract structured fields, classify documents, summarize sections, and expose all operations through FastAPI. Include asynchronous ingestion and persistent job status.

A strong portfolio should emphasize measurable outcomes. Instead of saying "implemented RAG," report the evaluation setup and what changed after improving chunking, retrieval, reranking, or prompting.

30. Final Engineering Principles

First, treat the model as a component, not the entire product.

Second, separate retrieval quality from generation quality. Debug the correct layer.

Third, use deterministic systems for deterministic tasks whenever practical. Database queries, calculators, permission checks, and business rules should not be delegated unnecessarily to an LLM.

Fourth, treat all external text as untrusted. This includes user messages, retrieved documents, web pages, and tool outputs.

Fifth, measure before optimizing. A change that sounds better can make retrieval or generation worse.

Sixth, prefer simple workflows when the problem is simple. Agentic behavior should earn its complexity through measurable value.

Seventh, design for failure. Model providers fail, databases become unavailable, tools time out, documents contain errors, and users ask questions outside the knowledge base.

Eighth, make the system observable. If you cannot see what was retrieved, what tools were called, and which model configuration was used, debugging becomes guesswork.

Ninth, version important artifacts. Prompt versions, model versions, evaluation datasets, document corpora, and retrieval configurations should be identifiable.

Tenth, build with security boundaries outside the model. Authentication, authorization, validation, rate limiting, and side-effect controls should be enforced by software.

Finally, learn to explain tradeoffs. A strong GenAI engineer is not defined by knowing the most framework names. The engineer is defined by the ability to turn an uncertain model into a useful, measurable, secure, maintainable software system.

31. Extended Technical Glossary

Agent: A system in which a model participates in iterative decision-making and action selection. Attention: A neural mechanism that computes relationships between token representations. Chunk: A retrievable unit of source content created during document splitting. Context window: The token capacity available to a model for a request according to its architecture and provider. Cross-encoder: A model that jointly evaluates a query and candidate text, often used for reranking. Embedding: A numerical vector representation used for similarity or downstream machine-learning tasks. Fine-tuning: Additional training that modifies model parameters using task-specific data. Function calling: A mechanism in which a model emits structured requests to invoke application-defined tools. Grounding: Constraining an answer to supported evidence or trusted external data. Hallucination: Unsupported or fabricated model output. Hybrid search: Retrieval that combines lexical and semantic signals. LLM: Large language model trained to model token sequences at large scale. Metadata: Information describing a document or chunk, often used for filtering and traceability. MMR: Maximum Marginal Relevance, a method that balances relevance and diversity in retrieval. Prompt injection: An attack in which untrusted content attempts to alter model behavior. RAG: Retrieval-Augmented Generation, a pattern that supplies retrieved external information to a generative model. Reranking: A second-stage process that reorders retrieved candidates using a more precise scoring model. Retriever: A component that selects candidate documents or chunks for a query. Semantic search: Search based primarily on meaning represented through learned embeddings. Token: A unit used by a model's tokenizer to represent text. Tool: An application-defined operation that a model can request through a structured interface. Vector database: A storage and retrieval system designed to search numerical vectors efficiently.

32. RAG Test Question Bank

The following questions are intentionally varied so they can be used as a test set for a RAG project.

1. What is the difference between an embedding and a language model? 2. Why can a low-temperature generation still hallucinate? 3. What is the purpose of chunk overlap? 4. Why can excessive chunk overlap hurt retrieval quality? 5. What is the difference between vector search and keyword search? 6. When is hybrid search useful? 7. What does a reranker do? 8. Why cannot a reranker recover a document that was not retrieved? 9. What is MMR and why can it improve context diversity? 10. Why should metadata be stored with chunks? 11.

What is the difference between RAG and fine-tuning? 12. Why should authorization filtering happen outside the LLM? 13. What is prompt injection? 14. Why are tool arguments untrusted? 15. What makes an agent different from a deterministic workflow? 16. Why can a smaller model sometimes be a better production choice? 17. What is retrieval recall at k? 18. How can you determine whether a RAG failure is a retrieval problem or a generation problem? 19. Why are citations not automatically proof of factual correctness? 20. What information should a production RAG trace contain? 21. How can caching introduce a security problem? 22. Why should side-effecting tools require stronger controls? 23. What is context engineering? 24. Why is more context not always better? 25. How does tokenization affect cost? 26. What is the role of FastAPI in a GenAI architecture? 27. Why should document ingestion be idempotent? 28. How should an enterprise system handle document versions? 29. What should be tested when an agent's tool call fails? 30. What makes a GenAI portfolio project demonstrate engineering maturity?

For stronger evaluation, ask multi-hop questions that require evidence from two or more sections. For example: "Explain why hybrid retrieval followed by reranking can improve an enterprise RAG system, and describe one limitation of this architecture." Another useful test is a contradiction question that asks the model to distinguish two related concepts rather than merely retrieve a definition.

33. Suggested RAG Evaluation Dataset Design

A serious RAG benchmark should contain several categories.

Category A: Direct factual retrieval. The answer is explicitly stated in one chunk.

Category B: Paraphrase retrieval. The question uses different words from the source.

Category C: Multi-chunk synthesis. The answer requires combining information from multiple sections.

Category D: Negative retrieval. The corpus does not contain enough evidence, so the expected behavior is to acknowledge insufficiency.

Category E: Metadata filtering. The answer requires selecting the correct document version or category.

Category F: Distractor retrieval. Several chunks contain related words but only one contains the required evidence.

Category G: Security. The test asks whether a user can access restricted information or whether a prompt attempts to override system instructions.

Category H: Comparative reasoning. The answer must distinguish two concepts such as fine-tuning and RAG, or workflow and agent.

For each test question, record the expected evidence. This allows retrieval evaluation independently of generation evaluation.

A useful dataset record contains: question, expected_answer, relevant_document_ids, relevant_chunk_ids, metadata_filters, question_category, difficulty, and notes about acceptable alternative answers.

When evaluating a model-as-judge, include examples where the model must say that evidence is insufficient. Otherwise a judge can unintentionally reward fluent but unsupported answers.

34. Conclusion: What a GenAI Engineer Really Builds

The most important shift is to stop thinking of Generative AI as simply "asking a model questions." Professional GenAI engineering is the discipline of building systems around models so that those systems can solve real problems under real constraints.

The engineer must understand data preparation because bad source data creates bad retrieval. The engineer must understand embeddings because semantic search depends on representation quality. The engineer must

understand chunking because retrieval units determine what evidence can be found. The engineer must understand reranking because candidate ordering affects context quality. The engineer must understand prompt and context engineering because retrieved information must be presented clearly. The engineer must understand model behavior because generation is probabilistic. The engineer must understand APIs, databases, authentication, Docker, testing, observability, and security because a model call is not a production application.

The strongest GenAI systems are therefore layered systems: trusted data, structured ingestion, retrieval, ranking, context engineering, generation, validation, security, observability, evaluation, and continuous improvement.

If you can build, measure, debug, and explain such a system, you are operating at the level expected of a serious GenAI engineer. The goal is not to memorize every framework. The goal is to understand the underlying principles well enough to select the right architecture, identify failure modes, and improve the system with evidence.

This document can serve as a knowledge corpus for that learning process and as a deliberately varied test corpus for your RAG project. Use it not only to ask "what is X?" but to test retrieval recall, semantic matching, metadata filtering, multi-document synthesis, contradiction handling, citation accuracy, refusal behavior, and grounded generation.

Appendix A. Detailed RAG Failure Analysis

Consider a RAG application that receives the question "What is the maximum number of agent iterations allowed by our architecture?" but returns an unrelated paragraph about token limits. Several failures could produce this outcome.

If the correct document was never indexed, the issue is ingestion. If it was indexed but split into an awkward chunk where the relevant sentence lost its heading and context, the issue may be chunking. If the chunk exists but its embedding does not make it competitive with distractors, the issue may be embedding or query formulation. If the correct chunk appears at rank 12 but the system keeps only five results, the issue is retrieval recall or top-k selection. If it appears among the candidates but is ranked below similar passages, reranking may help. If the correct chunk reaches the final prompt but the model ignores it, context formatting or generation behavior may be involved. If the model answers from prior knowledge despite the context, grounding controls and evaluation should expose the behavior.

This layered analysis prevents random changes. A developer should not immediately replace the LLM when the real problem is that the correct chunk was never retrieved.

Another failure mode is context pollution. Suppose retrieval returns ten chunks, eight of which discuss general agent design while two contain the exact answer. The model may synthesize a broad response and omit the exact limit. Reducing k, reranking candidates, or applying context compression may improve the result.

Contradictory documents create another challenge. If one document says the iteration limit is five and a newer approved document says it is ten, the system needs document-version metadata and a policy for authority. Simply giving both chunks to an LLM and asking it to decide is weaker than representing document priority explicitly.

These examples demonstrate a core principle: RAG quality is an end-to-end property, not a single-model property.

Appendix B. Example Production Request Lifecycle

A user sends a request to a GenAI application. The client attaches an authentication token. The API validates the token and determines the user's permissions and tenant.

The server validates the request body, applies rate limits, and creates a request ID. The query is normalized without changing its meaning. If the system uses query rewriting, the rewritten query is recorded separately from the original question.

The retriever creates a query embedding and performs vector search. In a hybrid architecture, keyword search may execute in parallel. The candidate sets are merged. Metadata filters ensure that only authorized documents are eligible.

A reranker scores the candidate chunks. The context builder removes duplicates, preserves useful metadata, and selects a context set that fits the model's token budget.

The prompt contains system instructions, the user's question, and clearly delimited evidence. The model generates an answer. If tools are available, the model may request a tool. The server validates the tool request, checks authorization, executes the tool, and returns the structured result to the model.

Before the response reaches the user, the application may validate its structure, inspect citations, apply content policies, and record evaluation signals.

The server then streams or returns the final answer. Logs contain the request ID and non-sensitive operational metadata. Metrics record latency, token usage, retrieval statistics, and errors.

This lifecycle shows why a GenAI engineer needs both AI knowledge and conventional software engineering skills.

Appendix C. Advanced Retrieval Design

Advanced retrieval systems often use more than one retrieval representation. A dense retriever captures semantic relationships, while a sparse retriever captures lexical overlap. A hybrid system can run both and combine their candidate sets before reranking.

Reciprocal Rank Fusion is one way to combine rankings without requiring the scores from different retrievers to be directly comparable. The method rewards documents that appear near the top of multiple rankings. Other systems normalize scores and use weighted combinations. The exact approach should be validated on the application's benchmark.

Query rewriting can improve retrieval when users ask conversational questions. For example, after a conversation about database indexing, the user might ask "what about the second option?" A query rewriter can transform that into a self-contained retrieval query. The original user question should still be preserved for final response generation.

HyDE-style approaches generate a hypothetical answer or document representation and use that generated representation for retrieval. Such methods can help some semantic retrieval tasks but also introduce model-generated assumptions into the retrieval path. They should therefore be benchmarked rather than adopted automatically.

Multi-query retrieval generates several query variants to increase recall. This can be useful for ambiguous natural-language questions but increases retrieval cost. Deduplication becomes important when multiple queries return the same chunks.

Parent-child retrieval stores small searchable units while returning a larger parent section for generation. This can provide precise matching without sacrificing broader context. Another approach is sentence-window retrieval, where a highly relevant sentence is identified first and neighboring sentences are added before generation.

Hierarchical retrieval can first identify a document, then a section, then a chunk. This is useful for large enterprise corpora where global retrieval over every chunk is expensive or noisy.

These techniques demonstrate that "vector database search" is only one design point within a larger information-retrieval architecture.

Appendix D. Advanced Agent Architecture and Guardrails

A robust agent architecture separates decision-making from execution. The model can propose an action, but the application decides whether that action is permitted. This distinction is critical when actions have external side effects.

A useful pattern is propose -> validate -> authorize -> execute -> observe. The proposal is model generated. Validation checks schema and argument constraints. Authorization checks identity and policy. Execution calls the real system. Observation returns a structured result.

An agent should have a termination condition. Possible conditions include successful completion, a maximum step count, a timeout, repeated identical actions, an unrecoverable tool error, or a human escalation.

Loop detection matters because an agent can repeatedly call the same tool with slightly different arguments without making progress. The application can record recent actions and observations and detect repetition.

Tool results should be structured. A tool returning a large block of unstructured text makes it harder for the model to reason reliably and increases context usage. A structured result might contain status, records, warnings, and an error code.

Human approval can be inserted before sensitive actions. For example, an agent may prepare an email and show it to the user before sending it. It may prepare a refund request before a human authorizes the transaction.

Agent memory should also have boundaries. Storing every conversation indefinitely can create privacy, cost, and retrieval problems. Memory should have a purpose, retention policy, access policy, and deletion mechanism.

The strongest agent systems are often not the most autonomous. They are the systems whose autonomy is bounded precisely enough that users can trust the behavior.

Appendix E. Data Governance and Knowledge-Base Lifecycle

A knowledge base should have an ownership model. Every important document should have an identifiable source, owner, version, status, and lifecycle state.

A useful lifecycle is draft -> reviewed -> approved -> published -> superseded -> archived. Retrieval should generally prefer the correct current state rather than treating every document as equally valid.

Document ingestion should record provenance. Provenance answers questions such as where the text came from, when it was extracted, which parser version was used, and which embedding model created the vector.

When the embedding model changes, old and new vectors should not be mixed casually. A new index version can be built and evaluated before it becomes active.

Blue-green indexing can reduce risk. One index serves production while another is built and evaluated. After validation, traffic can switch to the new index. If quality degrades, the previous index remains available for rollback.

Data deletion requirements should also reach derived representations. Removing a source document may require deleting its chunks, embeddings, cached retrieval results, and generated artifacts that retained the content.

Access control should be applied consistently at ingestion and retrieval. A common mistake is to attach an access-control label to a document but forget to enforce it during search. Security metadata has value only when it participates in an actual authorization decision.

Knowledge-base quality should be periodically audited. Duplicate content, obsolete policies, broken links, parsing failures, and missing metadata can silently degrade retrieval quality over time.

Appendix F. Cost Modeling for a GenAI Application

A useful cost model starts with the number of requests and the average tokens processed per request. Input tokens may include system instructions, conversation history, retrieved context, and the user question. Output tokens are generated by the model.

Suppose a service handles 10,000 requests per month. If each request sends 3,000 input tokens and generates 500 output tokens, the monthly token volume can be estimated before provider-specific pricing is applied. Retrieval, embedding, storage, and reranking costs must then be added.

Cost optimization should focus on the largest contributors. If input context dominates, improve retrieval precision and context compression. If generation dominates, consider a smaller model for simple requests or shorter outputs. If embeddings dominate ingestion cost, cache embeddings and avoid reprocessing unchanged content.

A routing architecture can select models based on task difficulty. A lightweight model may classify requests and handle simple transformations, while a stronger model handles complex synthesis.

Caching repeated semantic questions can also reduce cost, but the cache must respect data permissions and document freshness.

Cost per successful task is often more meaningful than cost per API call. A very cheap model that fails frequently can produce a higher effective cost because users retry, agents execute more steps, or humans must correct outputs.

Performance and cost are connected. A slow model may reduce throughput and increase infrastructure requirements. A retrieval improvement that reduces generation context can simultaneously reduce latency and cost while improving answer quality.

Appendix G. System Design Interview Framework

When answering a GenAI system-design interview question, begin by clarifying the task and constraints. Ask about users, scale, data sensitivity, update frequency, latency expectations, model requirements, and acceptable failure behavior.

Next, define the data flow. Identify where source documents live, how they are ingested, how knowledge is indexed, how a query is retrieved, and how the answer is generated.

Then define APIs and storage. Describe authentication, authorization, relational data, object storage, vector storage, and observability.

Then discuss retrieval. Explain chunking, embeddings, metadata filters, hybrid search, top-k, reranking, and context assembly. Do not simply say "use a vector database."

Then discuss generation. Identify the model, prompt strategy, structured output, streaming behavior, and grounding approach.

Then discuss reliability. Cover timeouts, retries, fallbacks, provider outages, malformed documents, empty retrieval results, and model errors.

Then discuss security. Cover tenant isolation, prompt injection, secret management, tool permissions, output validation, and audit logs.

Finally, discuss evaluation and scaling. Explain the benchmark, retrieval metrics, answer metrics, load testing, caching, indexing strategy, and deployment approach.

The strongest interview response is not the one with the longest technology list. It is the one that makes assumptions explicit, identifies tradeoffs, and explains how the design would be measured.

Appendix H. Practical Engineering Checklist

Before calling a GenAI project production-ready, verify the following.

Data: Every source has a known owner and provenance. Parsing failures are detectable. Duplicate documents are controlled. Document versions are represented. Sensitive fields are classified.

Ingestion: Chunking is deterministic or reproducible. Chunk metadata is preserved. Embedding model versions are tracked. Indexing can be repeated without uncontrolled duplication. Failed ingestion jobs are visible.

Retrieval: Representative questions exist. Recall at k is measured. Hybrid retrieval has been considered where exact identifiers matter. Metadata filters are tested. Reranking has a measured benefit before being retained. Duplicate chunks are controlled.

Generation: Prompts are versioned. Output schemas are validated. Grounding behavior is tested. Unsupported questions have defined behavior. Citations are checked for correctness.

Tools and agents: Tool schemas are strict. Arguments are validated server-side. Authorization occurs before execution. Side effects have confirmation or policy controls. Iteration limits exist. Timeouts exist. Loops and repeated failures are detected.

Backend: Authentication is implemented. Authorization is enforced. Rate limiting exists. Secrets are not stored in source code. API errors are structured. Streaming is tested if used.

Operations: Logs have request IDs. Sensitive content is handled according to policy. Latency is measured by component. Token usage and cost are measured. Health checks exist. Deployment can roll back.

Evaluation: The test set contains positive and negative questions. Retrieval and generation are evaluated separately. Adversarial cases are included. Regression tests run after meaningful changes. Production incidents become new evaluation cases.

DOCUMENT METADATA AND RAG TESTING NOTES

Approximate source word count: 10,161. No images or diagrams are included. The corpus intentionally contains related concepts, distractors, multi-hop material, and explicit failure-analysis content so a RAG system can be tested beyond simple keyword matching.

Recommended evaluation dimensions: retrieval recall, ranking quality, context relevance, groundedness, answer relevance, citation correctness, negative-question behavior, metadata filtering, and security boundary enforcement.